

Step 1 — A Classification Chain With `RunnableBranch` — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume a `.env` with `OPENAI_API_KEY` set, and are run from inside `project_13_langchain_patterns/`.

Basic Version

Approach 1 — plain string category, no `RunnableBranch` yet

```
# classifier.py (Basic -- proves the classification idea before adding routing)
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

CLASSIFY_PROMPT = ChatPromptTemplate.from_template(
    "Classify this support ticket into exactly one word: "
    "billing, technical, or general.\n\nTicket: {ticket}"
)

def build_classification_chain():
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return CLASSIFY_PROMPT | model | StrOutputParser()

chain = build_classification_chain()
print(chain.invoke({"ticket": "I was charged twice for my subscription."}))
```

Expected output:

```
billing
```

This proves the classifier itself works, but the category is a raw string you'd have to check by hand (`if category.strip().lower() == "billing":`), and there's no routing at all yet — that's what `RunnableBranch` adds next.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — structured classification, plus real `RunnableBranch` routing

```
# models.py
from pydantic import BaseModel, Field

class TicketClassification(BaseModel):
    category: str = Field(description="One of: billing, technical, general")
```

```
# classifier.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from models import TicketClassification

CLASSIFY_PROMPT = ChatPromptTemplate.from_template(
    "Classify this support ticket into exactly one category: "
    "billing, technical, or general.\n\nTicket: {ticket}"
)

def build_classification_chain():
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return CLASSIFY_PROMPT | model.with_structured_output(TicketClassification)
```

```
# handlers.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o-mini", temperature=0.3)

billing_prompt = ChatPromptTemplate.from_template(
    "You are a billing support agent. Be precise about charges, refunds, "
    "and invoices. Ticket: {ticket}"
)

technical_prompt = ChatPromptTemplate.from_template(
    "You are a technical support agent. Ask a clarifying troubleshooting "
```

```

    "question if needed. Ticket: {ticket}"
)
general_prompt = ChatPromptTemplate.from_template(
    "You are a friendly general support agent. Ticket: {ticket}"
)

billing_chain = billing_prompt | model | StrOutputParser()
technical_chain = technical_prompt | model | StrOutputParser()
general_chain = general_prompt | model | StrOutputParser()

```

```

# router.py
from langchain_core.runnables import RunnableBranch, RunnableParallel, RunnablePassthrough
from classifier import build_classification_chain
from handlers import billing_chain, technical_chain, general_chain

def is_billing(inputs):
    return inputs["category"].category == "billing"

def is_technical(inputs):
    return inputs["category"].category == "technical"

def build_router_chain():
    classification_chain = build_classification_chain()
    tag_with_category = RunnableParallel(
        ticket=RunnablePassthrough(),
        category=classification_chain,
    )
    router = RunnableBranch(
        (is_billing, billing_chain),
        (is_technical, technical_chain),
        general_chain,
    )
    return tag_with_category | router

```

```

# main.py
from router import build_router_chain

pipeline = build_router_chain()

tickets = {
    "billing": "I was charged twice for my Pro subscription this month, order INV",
    "technical": "The export button on the dashboard just spins forever and never"
}

```

```
"general": "Do you have a referral program?",  
}  
  
for label, text in tickets.items():  
    result = pipeline.invoke({"ticket": text})  
    print(f"[{label}] -> {result[:80]}...")
```

Expected output (shortened):

```
[billing] -> I'm sorry about the duplicate charge on order INV-2201...  
[technical] -> Thanks for reporting this. Could you tell me which browser...  
[general] -> Yes! We'd love for you to refer us...
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Difference from Basic: Basic classifies but doesn't route — you'd need your own `if/elif` around the result. Intermediate uses `.with_structured_output(TicketClassification)` so the category is a real, checked field, and wires it into a real `RunnableBranch` through `RunnableParallel`, so routing happens inside the LCEL pipeline itself, not in a separate block of Python you write by hand.

Advanced Version

Approach 1 — the ambiguous ticket, made visible instead of silently absorbed

```
# main.py -- add this to the loop above
ambiguous_ticket = (
    "My invoice looks wrong and the numbers on the dashboard don't match "
    "it either -- is this a billing mistake or is the dashboard broken?"
)

classification_chain = build_classification_chain()
classification = classification_chain.invoke({"ticket": ambiguous_ticket})
print("Classifier picked:", classification.category)

result = pipeline.invoke({"ticket": ambiguous_ticket})
print("Router sent it to:", result[:80])
```

Expected output (will vary run to run -- that's the point):

```
Classifier picked: billing
Router sent it to: I'm sorry to hear your invoice looks incorrect...
```

Run this a few times, or reword the ticket slightly. The classifier will confidently pick *one* category every time, even though the ticket itself is genuinely 50/50. `RunnableBranch` has no way to express "I'm 55% sure this is billing" — it only ever sees a single final answer and commits fully to one path. That's not something you can fix by adding a 4th branch; a fixed-branch router structurally cannot carry uncertainty forward.

Approach 2 — carrying the ambiguity forward instead of dropping it

```
# classifier.py -- an honest upgrade: ask the model to also flag ambiguity
from pydantic import BaseModel, Field
```

```

from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

class TicketClassification(BaseModel):
    category: str = Field(description="One of: billing, technical, general")
    is_ambiguous: bool = Field(
        description="True if the ticket genuinely fits more than one category"
    )

CLASSIFY_PROMPT = ChatPromptTemplate.from_template(
    "Classify this support ticket into exactly one category: "
    "billing, technical, or general. Also say whether the ticket genuinely "
    "reads like it could fit more than one category.\n\nTicket: {ticket}"
)

def build_classification_chain():
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return CLASSIFY_PROMPT | model.with_structured_output(TicketClassification)

```

Expected output for the ambiguous ticket:

```
TicketClassification(category='billing', is_ambiguous=True)
```

This doesn't stop `RunnableBranch` from picking exactly one path (it still has to — that's what a branch does) but it does stop the ambiguity from disappearing. `is_ambiguous` rides along in the `category` field of the tagged dict, ready for Step 4's `needs_human_review` to actually use it, instead of being computed once here and thrown away.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's router works correctly on every clearly-worded ticket, but it silently forces a single answer on a genuinely unclear one, the same way any fixed-branch router does. Approach 1 just proves that limit is real, with an actual run. Approach 2 doesn't remove the limit — it can't, `RunnableBranch` will always fully commit — but it stops the useful signal ("this one was genuinely unclear") from being thrown away the moment the branch is chosen.

Which one should you actually write? Approach 2's shape (the classifier also reporting its own uncertainty) — it costs one extra boolean field and pays for itself the first time someone asks "why did an obviously confusing ticket get sent straight to the customer with no review." A

`RunnableBranch` will never be able to loop back and ask the classifier "wait, are you sure?" — that specific capability (a decision that can revisit an earlier step) is exactly the kind of thing [Doc09's](#)

[comparison topic](#) says would need LangGraph, not LangChain. For this project, carrying the uncertainty forward as data is the right-sized fix — it doesn't need a graph to do it.